Continuation-Passing Style

Making the rest of the computation explicit

CS413

Fall 2026

A computation has a result and a next step

Consider evaluating `(2 + 3) * 4`. After computing `2 + 3`, the program still has work to do: multiply that result by 4.

`2 + 3` then `multiply by 4`

Continuation

A continuation is a function that says what to do with a value when that value becomes available.

Here the continuation for the sum is `fun x -> x * 4`. The final answer is obtained by giving it the value 5.

Direct style hides the next step

```
fun squarePlusOne(n) = n * n + 1
```

A direct-style function returns its answer to its caller. The caller's remaining work sits outside the function.

`squarePlusOne(3)` returns `10`

CPS changes the interface: instead of returning a result, a function receives a continuation and passes its result to that continuation.

A function in CPS receives its continuation

Direct style

```
squarePlusOne(n) = n*n + 1
```

Result type: `int`

Continuation-passing style

```
squarePlusOne_cps(n, k) =
  k(n*n + 1)
```

Result type: `(int -> R) -> R`

The identity continuation

`fun x -> x` asks for the final value unchanged. Passing it in recovers the ordinary result: `squarePlusOne_cps(3, id) = 10`.

The type variable `R` describes the eventual answer produced by the continuation; it need not be an integer.

Translate an expression by naming its next step

To translate `e` in a context that expects its value, write `CPS(e, k)`.

```
CPS(42, k) = k(42)
CPS(x,  k) = k(x)
```

For an addition, evaluate the left operand, then the right, then continue:

```
CPS(e1 + e2, k) =
  CPS(e1, fun v1 ->
    CPS(e2, fun v2 ->
      k(v1 + v2)))
```

The nested functions remember the unfinished work. This translation makes left-to-right evaluation order explicit.

Example: CPS for a nested arithmetic expression

Translate `(a + b) * c`:

```
CPS(a + b, fun sum ->
  CPS(c, fun factor ->
    k(sum * factor)))
```

1. Evaluate `a + b`; call the continuation with `sum`.
2. Evaluate `c`; call the next continuation with `factor`.
3. Multiply and hand the answer to `k`.

With `a=2`, `b=3`, `c=4`, and identity `k`, the result is 20.

Recursive calls must also pass the rest of the work

In factorial, after the recursive call returns, we still multiply by `n`. CPS stores that multiplication in a continuation:

```
fact_cps(n, k) =
  if n <= 1 then k(1)
  else fact_cps(n - 1,
    fun r -> k(n * r))
```

- Base case: the answer is 1, so pass 1 to `k`.
- Recursive case: when `fact(n-1)` produces `r`, multiply by `n`, then continue.
- Start with `fact_cps(4, fun x -> x)`; the result is 24.

The continuation records pending multiplications

For `fact_cps(4, id)`, the recursive descent builds this chain:

$$\texttt{id} \leftarrow \texttt{fun r -> id(4*r)} \leftarrow \texttt{fun r -> k4(3*r)} \leftarrow \cdots$$

The full continuation eventually represents

```
fun r -> id(4 * (3 * (2 * (1 * r))))
```

What changed?

Direct style leaves pending work in the call stack. CPS represents that work as functions passed explicitly from call to call.

(The displayed chain is schematic: each continuation closes over the values needed for its multiplication.)

Conditionals and function calls have CPS rules too

```
CPS(if test then yes else no, k) =
  CPS(test, fun b ->
    if b then CPS(yes, k)
         else CPS(no, k))
```

Only the chosen branch runs, and both branches receive the same continuation. A call `f(x)` first computes the function and argument, then applies the function to the argument and continuation:

```
CPS(f(x), k) = CPS(f, fun vf ->
  CPS(x, fun vx -> vf(vx, k)))
```

The exact rule depends on the language's evaluation order and calling convention.

CPS changes function types

A direct function returning `A` and accepting `B` has type

```
B -> A
```

Its CPS version accepts a continuation from `A` to some answer type `R`:

```
B -> (A -> R) -> R
```

- `A`: the value the direct function would have returned.
- `R`: the answer type chosen by the surrounding computation.

The continuation argument adds another function layer; polymorphic CPS may quantify over `R`.

CPS and tail calls

In `fact_cps`, the recursive call is the final action in its branch:

```
fact_cps(n - 1, continuation)
```

It is a tail call: the caller has no instructions left after it returns.

- Tail-call execution can reuse the current stack frame.
- CPS makes pending work explicit, but a continuation may still be a closure that uses memory.
- CPS by itself does not guarantee constant space; representation and runtime behavior matter.

CPS can represent more than ordinary returns

A function can choose which continuation to invoke:

- **Success and failure:** pass separate continuations for each outcome.
- **Exceptions:** send errors to an exceptional continuation.
- **Early exit:** invoke a continuation that skips the remaining work.
- **Control operators:** capture and later invoke a saved continuation.

These ideas all make control flow visible as data: functions that can be passed, stored, or selected.

Summary

- A continuation describes the rest of a computation.
- CPS passes that continuation explicitly instead of returning a value.
- Translation nests continuations around unfinished operations.
- CPS exposes control flow and makes calls tail calls, while closures may still consume space.

Quick check

In `fact_cps`, why does the recursive call receive `fun r -> k(n*r)` instead of `k` directly?

Optional example: Fibonacci in direct style

The Fibonacci function makes two recursive calls. The second call and the addition are work that remains after the first call returns:

```
fib(n) =
  if n <= 1 then n
  else fib(n - 1) + fib(n - 2)
```

For example, `fib(4)` computes `fib(3) + fib(2)`, or `2 + 1 = 3`. This definition returns its result normally; the caller does not see the pending second call or addition.

Optional example: Fibonacci in CPS

After `fib(n-1)` produces `left`, the function still needs to compute `fib(n-2)` and add the two results. The continuation stores that work:

```
fib_cps(n, k) =
  if n <= 1 then k(n)
  else fib_cps(n - 1, fun left ->
    fib_cps(n - 2, fun right ->
      k(left + right)))
```

Both recursive calls pass their results onward through continuations. With the identity continuation, `fib_cps(4, fun x -> x)` evaluates to 3.

Optional example: Fibonacci as lazy call events

Record both the entry to a call and its return. The continuation produces the remaining event stream after a recursive result is available:

```
event = Enter(n) | Return(value)
emit(event, rest) = lazy_cons(event, delay(rest))
fib_events(n, k) =
  emit(Enter(n),
    if n <= 1 then emit(Return(n), k(n))
    else fib_events(n - 1, fun left ->
      fib_events(n - 2, fun right ->
        emit(Return(left + right), k(left + right)))))
```

Start with `fib_events(4, fun result -> lazy_empty)`. The outer call emits `Enter(4)`; its recursive calls emit their own entry and return events before the outer call emits `Return(3)`. The tail is delayed, so events are produced as the consumer advances through the stream.

The event consumer drives the computation

Each invocation emits `Enter(n)` before its work and `Return(value)` after its work. The continuation keeps the pending second recursive call and addition, so returns appear in evaluation order.

Event stream

These are the only two event forms. The final event is the outer call's return, whose value is the Fibonacci result.

A real implementation needs a lazy-list type whose tail is suspended, such as a thunk or stream node.